

Analytics Programming with R

Chapter 8



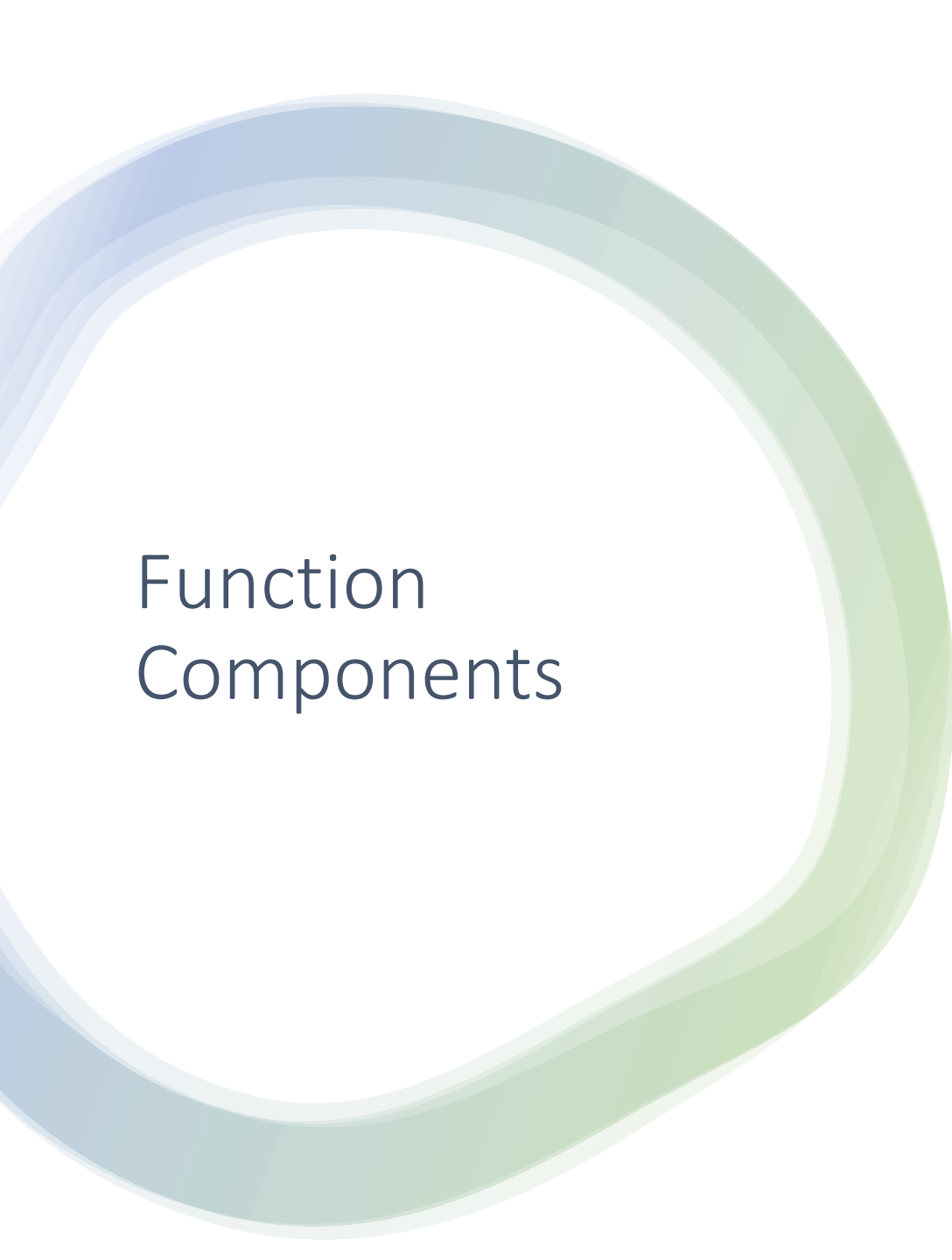
Functions

- A set of statements organized together to perform a specific task
- Functions are used to make code reusable and maintainable
- Functions can be built-in, or user defined



Functions

- In R, a function is an object so the R interpreter is able to pass control to the function, along with arguments that may be necessary for the function to accomplish the action
- The function in turn performs its task and returns control to the interpreter as well as any result which may be stored in other objects.



Function Components

- Function Name – The actual name of the function
- Arguments – a placeholder. When a function is invoked, you pass a value to an argument.
 - Arguments are optional. A function may contain no arguments.
 - Arguments can have default values
- Function Body - contains all the statements that define what the function does
- Return Value – the return value of a function is the last expression in the function to be evaluated



Writing Functions

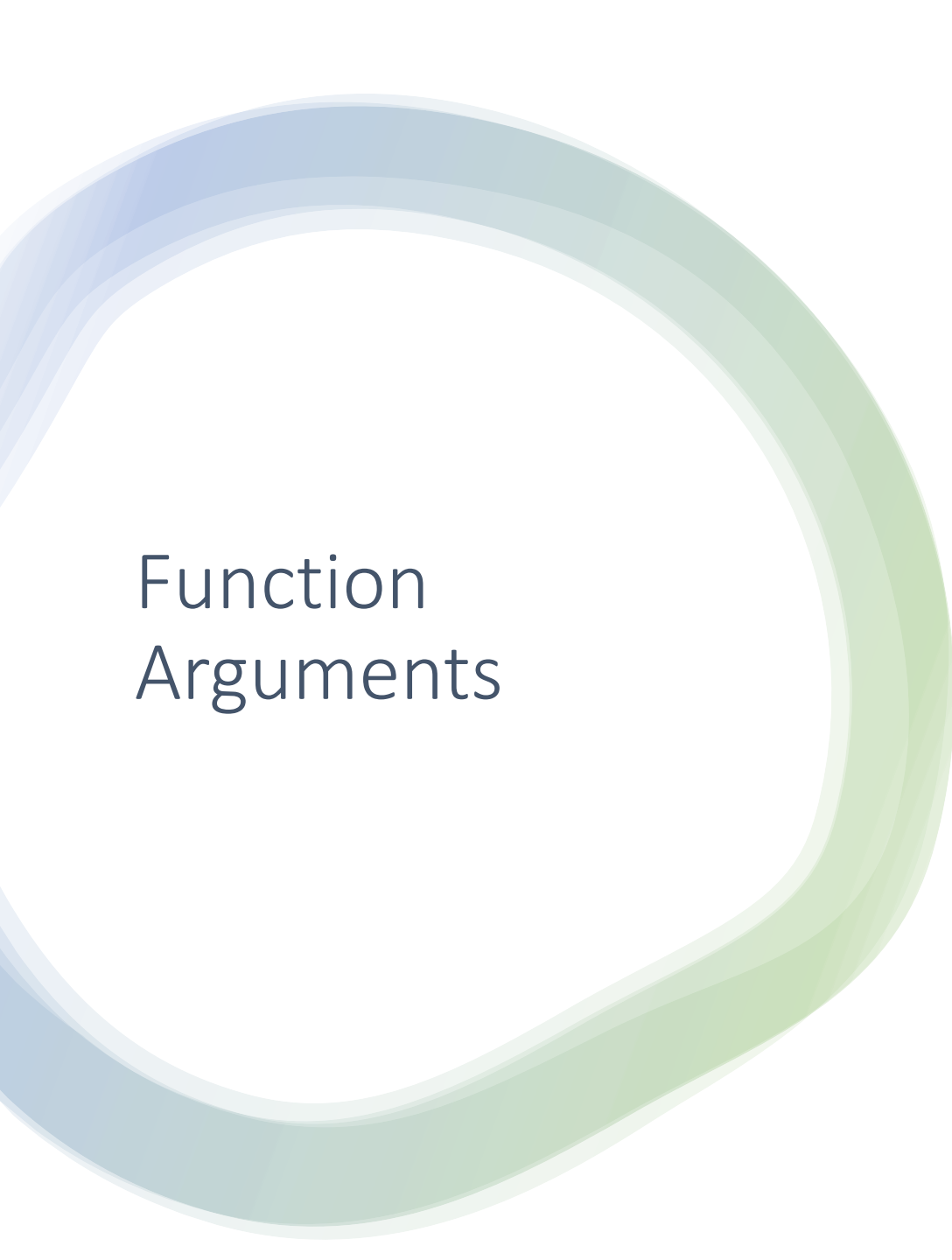
- Functions are assigned to objects like any other variable.
 - `newFunction <- function()`
- The parenthesis following a function can be empty or contain one or many arguments
 - Each argument in a function is separated by a comma
 - `newFunction <- function(x=1,"book2", FALSE)`



Writing Functions

- The body of the function must contain opening and closing braces.
 - `newFunction <- function()`
 {

 }
- The commands of the function are placed between the opening and closing braces.



Function Arguments

- Function arguments can have default arguments which are hard coded values
- The parenthesis following a function can be empty or contain one or many arguments
- Each argument in a function is separated by a comma
 - `newFunction <- function(x=1,"book2", FALSE)`



Calling Functions

- Functions are called using the function name followed by the open and close parenthesis.
 - Example function call: `newFunction()`
- The value of an argument can be supplied during a function call by either position or by name



Calling Functions

- Default arguments can be over written by supplying new values to pass in during the function call
- The "sprintf()" function allows you to pass arguments from the function call to the function within your scripts by using the special character of "%s"
- Arguments to functions are evaluated lazily, which means so they are evaluated only when needed by the function body.



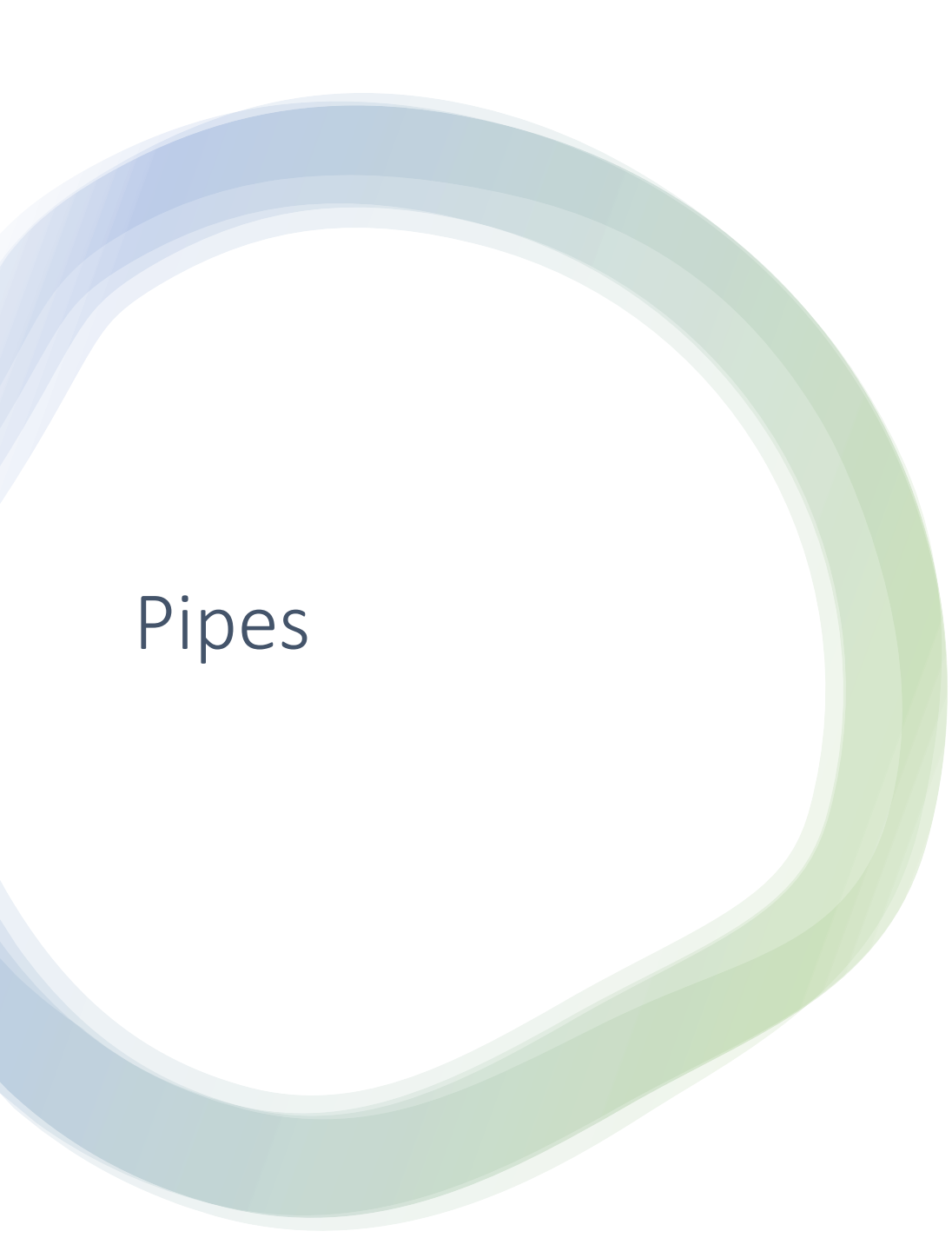
Return Values

- A function is written typically to return a value based off a calculation of some sorts.
 - $1 + 2 = 3$
 - 3 is the returned value of the addition of $1 + 2$
- Best practices in coding is that if a value is to be returned then it should explicitly specify that a value should be returned in the function body by using the `return()` command.



Return Values

- The return value of a function can be any R object. Although the return value is often a list, it could even be another function.
- You can transmit a value back to the caller by explicitly calling `return()`. Without this call, the value of the last executed statement will be returned by default.



Pipes

- Pipes are a new way to call functions in R
- Uses the “Magrittr” package
- Takes the value or object on the left side of the pipe and inserts it into the first argument of the function on the right side of the pipe.



Pipes

- Pipes are most useful for chaining a series of function calls together
 - `z <- c(1, 2, NA, 8, 3, NA, 3)` # A Vector with NA
`sum(is.na(z))` # Summing the number of NA's
[1] 2
- This can also be done using pipes.
 - `z %>% is.na %>% sum` # Summing the number of NA's
[1] 2